

DEEP LEARNING FOR FINANCE

MODULE 1 | LESSON 4

Multilayer Perceptron: Revisiting market Timing of AAPL stock with Hyper-parameter Tuning

Reading Time 60 minutes

Prior Knowledge Deep Learning, MLP, Timing Strategy, Hyper-parameter

Keywords MLP for Regression, Hyper-parameter Tuning, Keras Tuner , Hyperband

In this lesson we revisit the timing our investments in AAPL stock under a regression problem using MLPs with Hyper-parameter tuning. This way, we will be able to observe first-hand the power of Hyper-parameter optimization (HPO). In order to complicate the predictability problem a bit over the classification example we went over in Lesson 2, and also to conveniently use the simplest form of the Keras Tuner, our predictive model MLP will be a regression one. As usual, we will first focus on the Deep Learning side of the task by building a predictive regression model with HPO, and later on check the performance of such a trading strategy in the test sample.

1. Data and sources

As we did in the example of Lesson 2, we will keep using the **yfinance** library to retrieve daily Apple stock price data from Yahoo! Finance:

```
In [1]: import numpy as np
import pandas as pd

# Read the CSV file
file_path = 'AAPL.csv'
df = pd.read_csv(file_path, delimiter=';', decimal=',', parse_dates=['Date'], da
df = df.sort_values('Date')
df = df.set_index('Date')

# Rename '% Return' column to 'Ret'
df = df.rename(columns={'% Return': 'Ret'})

# Define the name variable
name = 'Ret'

# Set 'Date' as index and keep only 'Ret' column
df = df.dropna()
```

```
df.head()
```

Out[1]:

	Price	CVol	Change	% Change	Ret	Total Return (Gross)	Cumulative Return %	Open	High	L
Date										
1991-12-02	0.46	118899320.0	0.01	1.97	1.97	0.48	276,80	0.45	0.46	C
1991-12-03	0.45	102860910.0	-0.01	-2.42	-2.42	0.47	267,70	0.46	0.46	C
1991-12-05	0.45	99198490.0	0.00	-0.99	-0.99	0.47	264,06	0.45	0.46	C
1991-12-06	0.44	196985780.0	-0.01	-2.50	-2.50	0.46	254,96	0.44	0.44	C
1991-12-09	0.44	97832080.0	0.00	0.77	0.77	0.46	257,69	0.44	0.45	C

2. Timing Apple stock with Multilayer Perceptron (MLPs)

Now, let's start building our model in a similar fashion to what we have done in previous examples, starting with inputs and output variables:

2.1 Inputs and outputs

For, among others, comparability reasons, we will consider the very same inputs for the regression model as we did for the classification example of Lesson 2. Specifically, those are AAPL stock returns from the past 25, 60, 90, 120 and 240 days:

```
In [2]: df = df[['Ret']]

df["Ret25_i"] = df[name].rolling(25).apply(lambda x: 100 * (np.prod(1 + x / 100)
df["Ret60_i"] = df[name].rolling(60).apply(lambda x: 100 * (np.prod(1 + x / 100)
df["Ret90_i"] = df[name].rolling(90).apply(lambda x: 100 * (np.prod(1 + x / 100)
df["Ret120_i"] = df[name].rolling(120).apply(lambda x: 100 * (np.prod(1 + x / 100)
df["Ret240_i"] = df[name].rolling(240).apply(lambda x: 100 * (np.prod(1 + x / 100)

df = df.dropna()
df.tail(10)
```

Out[2]:

	Ret	Ret25_i	Ret60_i	Ret90_i	Ret120_i	Ret240_i
Date						
2025-04-23	2.43	-3.795072	-10.881083	-17.324816	-11.375229	13.166308
2025-04-24	1.84	-3.186662	-12.437332	-15.363482	-10.514112	14.812281
2025-04-25	0.44	-2.242569	-12.454764	-15.498093	-10.228100	15.098768
2025-04-28	0.41	-3.719238	-11.440489	-15.210988	-8.459465	14.426408
2025-04-29	0.51	-4.309509	-10.388438	-15.764123	-6.287033	15.809066
2025-04-30	0.61	-5.026928	-6.678198	-16.064459	-4.444495	14.500296
2025-05-01	0.39	-3.703195	-8.241178	-13.894452	-3.686575	14.238568
2025-05-02	-3.74	-8.267882	-11.549127	-17.690963	-7.887429	8.640630
2025-05-05	-3.15	-8.729653	-14.608582	-21.754709	-10.493604	5.155357
2025-05-06	-0.19	-10.636714	-12.675026	-22.144726	-12.535408	4.934575

- **Defining the output: Regression**

Then, we need to define our output variable. As we have already mentioned, we will face a regression problem here so our output is actually a continuous variable containing the returns of AAPL stock for some future time span.

In this case, our model will aim to predict the return of AAPL stock over the next **25 days**. Please note that this choice is completely subjective (the model may even work better for some other time span!). Once you have gone through the entire notebook and understood the basics of the process, please go ahead and play around with all these features!

```
In [3]: df["Ret25"] = df["Ret25_i"].shift(-25)
df = df.dropna()
df.tail(10)
```

Out[3]:

	Ret	Ret25_i	Ret60_i	Ret90_i	Ret120_i	Ret240_i	Ret25
Date							
2025-03-18	-0.61	-6.554621	-15.996749	-4.367412	-6.574936	25.694524	-3.795072
2025-03-19	1.20	-7.450848	-13.129686	-2.831145	-4.729781	28.099556	-3.186662
2025-03-20	-0.53	-9.595756	-14.190763	-3.970333	-5.612264	26.811931	-2.242569
2025-03-21	1.95	-9.613488	-14.131805	-1.773607	-3.346428	29.921378	-3.719238
2025-03-24	1.13	-9.738442	-13.429862	-2.744908	-2.750216	30.800886	-4.309509
2025-03-25	1.37	-8.456087	-13.241573	-1.294066	-1.536051	33.487222	-5.026928
2025-03-26	-0.99	-9.507160	-14.374483	-1.084266	-4.693366	31.220908	-3.703195
2025-03-27	1.05	-8.912227	-12.318013	-0.443875	-0.806104	34.087094	-8.267882
2025-03-28	-2.66	-11.237523	-13.499902	-4.411194	-3.685448	25.103400	-8.729653
2025-03-31	1.94	-10.082015	-11.191258	-1.163172	-1.333480	26.442997	-10.636714

In [4]: `df.describe()`

Out[4]:

	Ret	Ret25_i	Ret60_i	Ret90_i	Ret120_i	Ret240_i	
count	8010.000000	8010.000000	8010.000000	8010.000000	8010.000000	8010.000000	8
mean	0.115211	2.919923	7.325607	11.229095	15.196986	32.624228	
std	2.691975	13.389784	21.602310	27.567421	33.122047	56.101787	
min	-51.870000	-69.598132	-77.074930	-72.483802	-74.729250	-75.007329	
25%	-1.160000	-4.791231	-4.843004	-4.884834	-3.924481	-6.063950	
50%	0.100000	2.894122	7.114339	10.713840	14.713342	29.238560	
75%	1.370000	10.587134	19.263231	26.127322	30.736367	59.176953	
max	33.230000	121.286374	116.442841	135.727260	167.513049	300.683525	



2.2 Train-Test samples and Scaling

The next important step, as you know, is in defining our train and test samples, together with the scaling of variables.

Now, as you may remember, we did not perform any scaling in the example from Lesson 2, and that is because all of the input and output variables are essentially of the same nature (i.e., stock returns), but note that it is common (and good) practice to perform the scaling. You can incorporate the scaling here by relying on what we did in the last module of ML. Indeed, you can actually try check whether different scaling methods matter for the predictions.

As for the definition of test and train samples, we will devote the usual first 60% of examples in that data (in chronological order) to training and the remaining 40% to testing:

```
In [5]: # Train-test split
ts = int(0.4 * len(df)) # Number of observations in test sample (20%)
split_time = len(df) - ts # Start of test sample
test_time = df.index[split_time:] # Extract Date index for test period
Ret_vector = df['Ret'][split_time:].values # Extract Ret values for test period
df.tail()
```

```
Out[5]:
```

	Ret	Ret25_i	Ret60_i	Ret90_i	Ret120_i	Ret240_i	Ret25
Date							
2025-03-25	1.37	-8.456087	-13.241573	-1.294066	-1.536051	33.487222	-5.026928
2025-03-26	-0.99	-9.507160	-14.374483	-1.084266	-4.693366	31.220908	-3.703195
2025-03-27	1.05	-8.912227	-12.318013	-0.443875	-0.806104	34.087094	-8.267882
2025-03-28	-2.66	-11.237523	-13.499902	-4.411194	-3.685448	25.103400	-8.729653
2025-03-31	1.94	-10.082015	-11.191258	-1.163172	-1.333480	26.442997	-10.636714

So let's formally split our data in train and test samples for input and output variables:

```
In [6]: from sklearn.model_selection import train_test_split

Xdf, ydf = df.iloc[:, 1:-1], df.iloc[:, -1]
X = Xdf.astype("float32")
y = ydf.astype("float32")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=ts, shuffle=False
) # It is important to keep "shuffle=False"
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
(4806, 4) (3204, 4) (4806,) (3204,)
```

2.3 Model and Training

Finally, it is time to set up our model. This is where the new things in the lesson come, which mainly sum up to one: **Keras Tuner**.

- **Keras Tuner**

Keras Tuner is a library that helps us in Hyper-parameter tuning for our TensorFlow programs. The functioning of the library is fairly easy and intuitive, with some specific syntax that you should be aware of. Despite that we will be using the Keras Tuner in this and future notebooks, we strongly recommend that you take a look at some extra resources:

- This simple tutorial from the TensorFlow website:
https://www.tensorflow.org/tutorials/keras/keras_tuner?hl=en
- There are multiple online examples and tutorials on how to use the Keras Tuner, here you have one that you can consult for clarification:
<https://www.youtube.com/watch?v=6Nf1x7qThR8>
- Keras Tuner **library documentation**: https://keras.io/api/keras_tuner/

Now the Keras Tuners works in a rather simple way, but it requires us to perform tasks in an organized way. Let's see how.

First we will need to import the library, of course:

```
In [7]: !pip install keras_tuner
```

Requirement already satisfied: keras_tuner in /usr/local/lib/python3.11/site-packages (1.4.7)

Requirement already satisfied: keras in /usr/local/lib/python3.11/site-packages (from keras_tuner) (3.9.2)

Requirement already satisfied: packaging in /usr/local/lib/python3.11/site-packages (from keras_tuner) (23.1)

Requirement already satisfied: requests in /usr/local/lib/python3.11/site-packages (from keras_tuner) (2.32.3)

Requirement already satisfied: kt-legacy in /usr/local/lib/python3.11/site-packages (from keras_tuner) (1.0.5)

Requirement already satisfied: absl-py in /usr/local/lib/python3.11/site-packages (from keras->keras_tuner) (2.2.2)

Requirement already satisfied: numpy in /usr/local/lib/python3.11/site-packages (from keras->keras_tuner) (1.26.2)

Requirement already satisfied: rich in /usr/local/lib/python3.11/site-packages (from keras->keras_tuner) (14.0.0)

Requirement already satisfied: namex in /usr/local/lib/python3.11/site-packages (from keras->keras_tuner) (0.0.9)

Requirement already satisfied: h5py in /usr/local/lib/python3.11/site-packages (from keras->keras_tuner) (3.13.0)

Requirement already satisfied: optree in /usr/local/lib/python3.11/site-packages (from keras->keras_tuner) (0.15.0)

Requirement already satisfied: ml-dtypes in /usr/local/lib/python3.11/site-packages (from keras->keras_tuner) (0.5.1)

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/site-packages (from requests->keras_tuner) (3.1.0)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/site-packages (from requests->keras_tuner) (3.4)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/site-packages (from requests->keras_tuner) (1.26.15)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/site-packages (from requests->keras_tuner) (2022.12.7)

Requirement already satisfied: typing-extensions>=4.5.0 in /usr/local/lib/python3.11/site-packages (from optree->keras->keras_tuner) (4.13.2)

Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.11/site-packages (from rich->keras->keras_tuner) (3.0.0)

Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.11/site-packages (from rich->keras->keras_tuner) (2.15.1)

Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.11/site-packages (from markdown-it-py>=2.2.0->rich->keras->keras_tuner) (0.1.2)

WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: <https://pip.pypa.io/warnings/venv>

[notice] A new release of pip available: 22.3.1 -> 25.1

[notice] To update, run: `pip install --upgrade pip`

```
In [8]: import keras_tuner as kt
import tensorflow as tf
from keras_tuner import HyperModel
```

```
2025-05-07 07:46:50.146413: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-05-07 07:46:50.195517: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
2025-05-07 07:46:51.312736: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
```

Then, in addition to setting the random seed for replicability and the validation split for the use of early-stopping later on, we need to define a function **build()** within the **MLP_model** class.

This build function contains the different 'choices' we are going to ask the Keras Tuner to decide for us. In this first and simple example we are simply tuning 3 hyper-parameters:

- **Dropout:** we are letting the tuner choose between a value for dropout of 0.2 or 0.3.
- **Number of layers in the model:** in this case, we will let the tuner choose between 1 and 5 hidden layers in the model. We do this using the **hp.Int()** method.
- **Number of units in each hidden layer:** the tuner will choose between 1 (min_value) and 25 (max_value) units in each layer, with a step size of 5.

```
In [9]: tf.keras.backend.clear_session()
tf.random.set_seed(1234)
val_split = 0.2

class MLP_model(HyperModel):
    def build(self, hp):
        # We define a constant activation function of ReLU form. We will not be
        act_fun = "relu"

        # We do ask the Keras Tuner to choose whether is best to have a dropout
        n_dropout = hp.Choice("n_dropout", values=[0.20, 0.30])

        model = tf.keras.models.Sequential()

        # Now, we will use a loop to let the tuner choose the number of layers to
        for i in range(1, hp.Int("num_layers", 1, 5)):
            # Within this loop, we will also ask the tuner to decide the optimal
            model.add(
                tf.keras.layers.Dense(
                    units=hp.Int(
                        "units_dense_" + str(i), min_value=1, max_value=25, step
                    ),
                    activation=act_fun,
                )
            )
```

```

    )
    model.add(tf.keras.layers.Dropout(n_dropout))

    model.add(tf.keras.layers.Dense(units=1))

    # As was the case with the activation function, there is no tuning on th
    hp_lr = 1e-5
    adam = tf.keras.optimizers.Adam(learning_rate=hp_lr)
    model.compile(optimizer=adam, loss="mean_absolute_error")

    return model

```

Now, once we have defined our HyperModel (i.e., **MLP_model()** class), we can perform the tuning of the hyper-parameters. This is, essentially, another optimization algorithm (**Tuner**), for which we would also need to make some decisions:

- **Hypermodel:** we have already defined our hypermodel: `MLP_model()`
- **Objective:** what is the objective function to optimize? In this case we will minimize the loss function in the validation set.
- **Other parameters:** these include whether we overwrite results, fixing the random seed, or storage of results (convenient for large complex models).

But what about the **Tuner**? In this case, we will select the Hyperband tuner embedded in Keras, for which you can consult the documentation here:

https://keras.io/api/keras_tuner/tuners/hyperband/#hyperband-class

Arguably, you could easily have decided to use another kind of optimizer from the ones available in the Keras Tuner (https://keras.io/api/keras_tuner/tuners/). Feel free to come back here once you have completed the notebook and see what would happen under another tuner!

```

In [10]: import os

from keras import backend as K

# First, we clear the session just to make sure our seeds are correctly working
K.clear_session()

# Then, we call the model and perform the tuning:
hypermodel = MLP_model()
tuner = kt.Hyperband(
    hypermodel,
    objective=kt.Objective("val_loss", direction="min"),
    overwrite=True,
    max_epochs=10,
    seed=1234,
    directory=os.path.normpath("C:/"),
)

# Let's run the tuner! (Warning: this could take time)

```

```
tuner.search(X_train, y_train, validation_split=val_split)
best_hps = tuner.get_best_hyperparameters(num_trials=1)[0]
```

Trial 30 Complete [00h 00m 05s]
val_loss: 8.497085571289062

Best val_loss So Far: 7.930953025817871
Total elapsed time: 00h 01m 14s

In [11]: `best_hps.values`

```
Out[11]: {'n_dropout': 0.2,
          'num_layers': 3,
          'units_dense_1': 11,
          'units_dense_2': 21,
          'units_dense_3': 11,
          'units_dense_4': 21,
          'tuner/epochs': 10,
          'tuner/initial_epoch': 4,
          'tuner/bracket': 1,
          'tuner/round': 1,
          'tuner/trial_id': '0021'}
```

Now, we can use these tuned hyper-parameter values to train our model, together with a usual early-stopping criteria:

```
In [12]: model = tuner.hypermodel.build(best_hps)
         es = tf.keras.callbacks.EarlyStopping(
             monitor="val_loss", mode="min", verbose=1, patience=20, restore_best_weights
         )

         # fit the model
         model.fit(
             X_train,
             y_train,
             validation_split=0.2,
             epochs=200,
             batch_size=32,
             verbose=2,
             callbacks=[es],
         )
```

Epoch 1/200
121/121 - 1s - 8ms/step - loss: 60.7589 - val_loss: 55.2892
Epoch 2/200
121/121 - 1s - 4ms/step - loss: 60.5621 - val_loss: 54.1300
Epoch 3/200
121/121 - 0s - 2ms/step - loss: 57.7094 - val_loss: 53.0120
Epoch 4/200
121/121 - 0s - 3ms/step - loss: 57.6723 - val_loss: 51.8763
Epoch 5/200
121/121 - 0s - 2ms/step - loss: 56.7486 - val_loss: 50.7455
Epoch 6/200
121/121 - 0s - 3ms/step - loss: 54.7629 - val_loss: 49.6327
Epoch 7/200
121/121 - 0s - 2ms/step - loss: 54.0818 - val_loss: 48.5352
Epoch 8/200
121/121 - 0s - 3ms/step - loss: 52.7764 - val_loss: 47.4433
Epoch 9/200
121/121 - 0s - 3ms/step - loss: 52.4463 - val_loss: 46.3664
Epoch 10/200
121/121 - 0s - 2ms/step - loss: 51.0419 - val_loss: 45.2981
Epoch 11/200
121/121 - 0s - 2ms/step - loss: 49.1633 - val_loss: 44.2710
Epoch 12/200
121/121 - 0s - 2ms/step - loss: 48.2504 - val_loss: 43.2420
Epoch 13/200
121/121 - 0s - 3ms/step - loss: 47.6171 - val_loss: 42.2085
Epoch 14/200
121/121 - 0s - 2ms/step - loss: 46.6572 - val_loss: 41.1772
Epoch 15/200
121/121 - 0s - 3ms/step - loss: 46.3400 - val_loss: 40.1405
Epoch 16/200
121/121 - 0s - 2ms/step - loss: 44.4603 - val_loss: 39.1390
Epoch 17/200
121/121 - 0s - 2ms/step - loss: 43.9083 - val_loss: 38.1496
Epoch 18/200
121/121 - 0s - 2ms/step - loss: 42.4401 - val_loss: 37.1640
Epoch 19/200
121/121 - 0s - 3ms/step - loss: 42.3213 - val_loss: 36.1823
Epoch 20/200
121/121 - 0s - 2ms/step - loss: 41.1723 - val_loss: 35.2069
Epoch 21/200
121/121 - 0s - 3ms/step - loss: 40.4357 - val_loss: 34.2402
Epoch 22/200
121/121 - 0s - 2ms/step - loss: 39.6546 - val_loss: 33.2788
Epoch 23/200
121/121 - 0s - 3ms/step - loss: 37.9864 - val_loss: 32.3380
Epoch 24/200
121/121 - 0s - 2ms/step - loss: 36.7197 - val_loss: 31.4274
Epoch 25/200
121/121 - 0s - 3ms/step - loss: 36.6431 - val_loss: 30.4911
Epoch 26/200
121/121 - 0s - 2ms/step - loss: 35.4619 - val_loss: 29.5744
Epoch 27/200
121/121 - 0s - 2ms/step - loss: 35.1836 - val_loss: 28.6601
Epoch 28/200
121/121 - 0s - 3ms/step - loss: 33.9963 - val_loss: 27.7613
Epoch 29/200
121/121 - 0s - 2ms/step - loss: 33.8394 - val_loss: 26.8533
Epoch 30/200
121/121 - 0s - 2ms/step - loss: 32.4806 - val_loss: 25.9754

Epoch 31/200
121/121 - 0s - 2ms/step - loss: 32.4247 - val_loss: 25.0806
Epoch 32/200
121/121 - 0s - 3ms/step - loss: 30.8789 - val_loss: 24.2257
Epoch 33/200
121/121 - 0s - 2ms/step - loss: 30.3947 - val_loss: 23.3857
Epoch 34/200
121/121 - 0s - 2ms/step - loss: 29.3155 - val_loss: 22.5609
Epoch 35/200
121/121 - 0s - 2ms/step - loss: 29.9390 - val_loss: 21.7183
Epoch 36/200
121/121 - 0s - 2ms/step - loss: 28.6318 - val_loss: 20.9313
Epoch 37/200
121/121 - 0s - 3ms/step - loss: 28.5308 - val_loss: 20.1763
Epoch 38/200
121/121 - 0s - 2ms/step - loss: 27.4630 - val_loss: 19.4093
Epoch 39/200
121/121 - 0s - 2ms/step - loss: 27.7033 - val_loss: 18.6465
Epoch 40/200
121/121 - 0s - 3ms/step - loss: 26.6301 - val_loss: 17.8917
Epoch 41/200
121/121 - 0s - 2ms/step - loss: 26.6684 - val_loss: 17.1859
Epoch 42/200
121/121 - 0s - 2ms/step - loss: 25.9235 - val_loss: 16.4922
Epoch 43/200
121/121 - 0s - 3ms/step - loss: 25.6605 - val_loss: 15.8231
Epoch 44/200
121/121 - 0s - 2ms/step - loss: 25.1218 - val_loss: 15.2005
Epoch 45/200
121/121 - 0s - 2ms/step - loss: 24.3488 - val_loss: 14.6080
Epoch 46/200
121/121 - 0s - 2ms/step - loss: 24.7702 - val_loss: 14.0242
Epoch 47/200
121/121 - 0s - 3ms/step - loss: 24.3320 - val_loss: 13.4639
Epoch 48/200
121/121 - 0s - 2ms/step - loss: 24.0687 - val_loss: 12.9288
Epoch 49/200
121/121 - 0s - 3ms/step - loss: 23.4684 - val_loss: 12.5103
Epoch 50/200
121/121 - 0s - 2ms/step - loss: 23.1510 - val_loss: 12.1158
Epoch 51/200
121/121 - 0s - 3ms/step - loss: 22.9878 - val_loss: 11.7210
Epoch 52/200
121/121 - 0s - 2ms/step - loss: 22.4297 - val_loss: 11.4079
Epoch 53/200
121/121 - 0s - 3ms/step - loss: 22.7052 - val_loss: 11.1035
Epoch 54/200
121/121 - 0s - 2ms/step - loss: 22.3456 - val_loss: 10.7939
Epoch 55/200
121/121 - 0s - 2ms/step - loss: 22.7060 - val_loss: 10.5122
Epoch 56/200
121/121 - 0s - 2ms/step - loss: 21.8393 - val_loss: 10.2943
Epoch 57/200
121/121 - 0s - 3ms/step - loss: 21.8695 - val_loss: 10.1145
Epoch 58/200
121/121 - 0s - 2ms/step - loss: 21.8387 - val_loss: 9.9440
Epoch 59/200
121/121 - 0s - 2ms/step - loss: 21.2471 - val_loss: 9.7854
Epoch 60/200
121/121 - 0s - 3ms/step - loss: 21.5242 - val_loss: 9.6590

Epoch 61/200
121/121 - 0s - 2ms/step - loss: 21.3108 - val_loss: 9.5461
Epoch 62/200
121/121 - 0s - 3ms/step - loss: 20.8671 - val_loss: 9.4506
Epoch 63/200
121/121 - 0s - 2ms/step - loss: 20.9935 - val_loss: 9.3523
Epoch 64/200
121/121 - 0s - 2ms/step - loss: 20.7324 - val_loss: 9.2702
Epoch 65/200
121/121 - 0s - 3ms/step - loss: 21.1063 - val_loss: 9.2183
Epoch 66/200
121/121 - 0s - 2ms/step - loss: 20.9318 - val_loss: 9.1708
Epoch 67/200
121/121 - 0s - 2ms/step - loss: 20.6103 - val_loss: 9.1311
Epoch 68/200
121/121 - 0s - 3ms/step - loss: 20.2585 - val_loss: 9.1028
Epoch 69/200
121/121 - 0s - 2ms/step - loss: 20.4676 - val_loss: 9.0806
Epoch 70/200
121/121 - 0s - 2ms/step - loss: 20.4251 - val_loss: 9.0507
Epoch 71/200
121/121 - 0s - 3ms/step - loss: 20.2054 - val_loss: 9.0271
Epoch 72/200
121/121 - 0s - 2ms/step - loss: 20.3319 - val_loss: 9.0065
Epoch 73/200
121/121 - 0s - 2ms/step - loss: 20.0951 - val_loss: 8.9848
Epoch 74/200
121/121 - 0s - 2ms/step - loss: 19.8333 - val_loss: 8.9672
Epoch 75/200
121/121 - 0s - 3ms/step - loss: 19.8877 - val_loss: 8.9504
Epoch 76/200
121/121 - 0s - 2ms/step - loss: 20.1308 - val_loss: 8.9287
Epoch 77/200
121/121 - 0s - 2ms/step - loss: 20.1989 - val_loss: 8.9140
Epoch 78/200
121/121 - 0s - 2ms/step - loss: 19.7295 - val_loss: 8.9034
Epoch 79/200
121/121 - 0s - 2ms/step - loss: 19.5249 - val_loss: 8.8879
Epoch 80/200
121/121 - 0s - 2ms/step - loss: 19.8580 - val_loss: 8.8795
Epoch 81/200
121/121 - 0s - 2ms/step - loss: 19.3913 - val_loss: 8.8655
Epoch 82/200
121/121 - 0s - 2ms/step - loss: 19.5088 - val_loss: 8.8486
Epoch 83/200
121/121 - 0s - 3ms/step - loss: 19.7102 - val_loss: 8.8317
Epoch 84/200
121/121 - 0s - 2ms/step - loss: 19.0490 - val_loss: 8.8178
Epoch 85/200
121/121 - 0s - 2ms/step - loss: 19.2517 - val_loss: 8.8017
Epoch 86/200
121/121 - 0s - 2ms/step - loss: 19.4366 - val_loss: 8.7883
Epoch 87/200
121/121 - 0s - 2ms/step - loss: 19.2342 - val_loss: 8.7759
Epoch 88/200
121/121 - 0s - 2ms/step - loss: 19.0456 - val_loss: 8.7625
Epoch 89/200
121/121 - 0s - 3ms/step - loss: 19.2636 - val_loss: 8.7499
Epoch 90/200
121/121 - 0s - 3ms/step - loss: 18.8381 - val_loss: 8.7439

Epoch 91/200
121/121 - 0s - 2ms/step - loss: 18.7859 - val_loss: 8.7276
Epoch 92/200
121/121 - 0s - 3ms/step - loss: 18.7102 - val_loss: 8.7135
Epoch 93/200
121/121 - 0s - 2ms/step - loss: 18.8509 - val_loss: 8.7022
Epoch 94/200
121/121 - 0s - 3ms/step - loss: 18.4496 - val_loss: 8.6888
Epoch 95/200
121/121 - 0s - 2ms/step - loss: 18.6287 - val_loss: 8.6800
Epoch 96/200
121/121 - 0s - 3ms/step - loss: 18.4114 - val_loss: 8.6630
Epoch 97/200
121/121 - 0s - 2ms/step - loss: 18.7350 - val_loss: 8.6539
Epoch 98/200
121/121 - 0s - 3ms/step - loss: 18.5229 - val_loss: 8.6417
Epoch 99/200
121/121 - 0s - 3ms/step - loss: 18.1560 - val_loss: 8.6340
Epoch 100/200
121/121 - 0s - 3ms/step - loss: 18.3762 - val_loss: 8.6195
Epoch 101/200
121/121 - 0s - 2ms/step - loss: 18.4043 - val_loss: 8.6087
Epoch 102/200
121/121 - 0s - 2ms/step - loss: 18.0133 - val_loss: 8.5992
Epoch 103/200
121/121 - 0s - 2ms/step - loss: 18.4696 - val_loss: 8.5874
Epoch 104/200
121/121 - 0s - 3ms/step - loss: 18.5018 - val_loss: 8.5749
Epoch 105/200
121/121 - 0s - 3ms/step - loss: 18.1270 - val_loss: 8.5667
Epoch 106/200
121/121 - 0s - 2ms/step - loss: 18.0545 - val_loss: 8.5586
Epoch 107/200
121/121 - 0s - 2ms/step - loss: 18.1155 - val_loss: 8.5503
Epoch 108/200
121/121 - 0s - 2ms/step - loss: 17.6787 - val_loss: 8.5401
Epoch 109/200
121/121 - 0s - 2ms/step - loss: 17.8990 - val_loss: 8.5294
Epoch 110/200
121/121 - 0s - 2ms/step - loss: 17.8791 - val_loss: 8.5178
Epoch 111/200
121/121 - 0s - 2ms/step - loss: 17.7268 - val_loss: 8.5094
Epoch 112/200
121/121 - 0s - 2ms/step - loss: 17.8260 - val_loss: 8.4995
Epoch 113/200
121/121 - 0s - 2ms/step - loss: 17.5905 - val_loss: 8.4873
Epoch 114/200
121/121 - 0s - 2ms/step - loss: 17.6476 - val_loss: 8.4740
Epoch 115/200
121/121 - 0s - 2ms/step - loss: 17.5049 - val_loss: 8.4667
Epoch 116/200
121/121 - 0s - 2ms/step - loss: 17.4171 - val_loss: 8.4619
Epoch 117/200
121/121 - 0s - 3ms/step - loss: 17.4891 - val_loss: 8.4548
Epoch 118/200
121/121 - 0s - 3ms/step - loss: 17.2399 - val_loss: 8.4461
Epoch 119/200
121/121 - 0s - 2ms/step - loss: 17.7266 - val_loss: 8.4394
Epoch 120/200
121/121 - 0s - 2ms/step - loss: 17.1003 - val_loss: 8.4281

Epoch 121/200
121/121 - 0s - 2ms/step - loss: 17.1450 - val_loss: 8.4250
Epoch 122/200
121/121 - 0s - 3ms/step - loss: 17.2077 - val_loss: 8.4173
Epoch 123/200
121/121 - 0s - 2ms/step - loss: 17.1851 - val_loss: 8.4081
Epoch 124/200
121/121 - 0s - 2ms/step - loss: 17.0245 - val_loss: 8.4005
Epoch 125/200
121/121 - 0s - 2ms/step - loss: 17.0555 - val_loss: 8.3916
Epoch 126/200
121/121 - 0s - 2ms/step - loss: 17.3224 - val_loss: 8.3841
Epoch 127/200
121/121 - 0s - 2ms/step - loss: 17.0983 - val_loss: 8.3764
Epoch 128/200
121/121 - 0s - 3ms/step - loss: 17.1480 - val_loss: 8.3719
Epoch 129/200
121/121 - 0s - 2ms/step - loss: 16.9905 - val_loss: 8.3690
Epoch 130/200
121/121 - 0s - 2ms/step - loss: 17.1824 - val_loss: 8.3590
Epoch 131/200
121/121 - 0s - 3ms/step - loss: 16.8188 - val_loss: 8.3519
Epoch 132/200
121/121 - 0s - 2ms/step - loss: 16.7538 - val_loss: 8.3456
Epoch 133/200
121/121 - 0s - 2ms/step - loss: 16.7591 - val_loss: 8.3410
Epoch 134/200
121/121 - 0s - 3ms/step - loss: 16.6442 - val_loss: 8.3340
Epoch 135/200
121/121 - 0s - 2ms/step - loss: 16.8611 - val_loss: 8.3282
Epoch 136/200
121/121 - 0s - 2ms/step - loss: 16.4589 - val_loss: 8.3210
Epoch 137/200
121/121 - 0s - 2ms/step - loss: 16.5247 - val_loss: 8.3154
Epoch 138/200
121/121 - 0s - 2ms/step - loss: 16.1393 - val_loss: 8.3103
Epoch 139/200
121/121 - 0s - 2ms/step - loss: 16.7264 - val_loss: 8.3063
Epoch 140/200
121/121 - 0s - 3ms/step - loss: 16.4530 - val_loss: 8.3024
Epoch 141/200
121/121 - 0s - 2ms/step - loss: 16.5194 - val_loss: 8.2980
Epoch 142/200
121/121 - 0s - 2ms/step - loss: 16.5381 - val_loss: 8.2921
Epoch 143/200
121/121 - 0s - 2ms/step - loss: 16.4405 - val_loss: 8.2878
Epoch 144/200
121/121 - 0s - 2ms/step - loss: 16.3134 - val_loss: 8.2824
Epoch 145/200
121/121 - 0s - 3ms/step - loss: 16.5686 - val_loss: 8.2787
Epoch 146/200
121/121 - 0s - 2ms/step - loss: 16.0627 - val_loss: 8.2778
Epoch 147/200
121/121 - 0s - 2ms/step - loss: 16.2156 - val_loss: 8.2741
Epoch 148/200
121/121 - 0s - 3ms/step - loss: 16.4070 - val_loss: 8.2690
Epoch 149/200
121/121 - 0s - 3ms/step - loss: 16.3552 - val_loss: 8.2653
Epoch 150/200
121/121 - 0s - 2ms/step - loss: 16.4400 - val_loss: 8.2619

Epoch 151/200
121/121 - 0s - 2ms/step - loss: 16.3181 - val_loss: 8.2577
Epoch 152/200
121/121 - 0s - 2ms/step - loss: 15.9846 - val_loss: 8.2554
Epoch 153/200
121/121 - 0s - 2ms/step - loss: 15.9779 - val_loss: 8.2510
Epoch 154/200
121/121 - 0s - 2ms/step - loss: 15.9585 - val_loss: 8.2458
Epoch 155/200
121/121 - 0s - 2ms/step - loss: 15.8787 - val_loss: 8.2418
Epoch 156/200
121/121 - 0s - 2ms/step - loss: 15.7248 - val_loss: 8.2389
Epoch 157/200
121/121 - 0s - 2ms/step - loss: 15.9865 - val_loss: 8.2365
Epoch 158/200
121/121 - 0s - 3ms/step - loss: 15.9792 - val_loss: 8.2338
Epoch 159/200
121/121 - 0s - 2ms/step - loss: 15.9160 - val_loss: 8.2336
Epoch 160/200
121/121 - 0s - 2ms/step - loss: 15.8486 - val_loss: 8.2327
Epoch 161/200
121/121 - 0s - 3ms/step - loss: 15.9467 - val_loss: 8.2316
Epoch 162/200
121/121 - 0s - 2ms/step - loss: 15.6031 - val_loss: 8.2291
Epoch 163/200
121/121 - 0s - 2ms/step - loss: 15.5325 - val_loss: 8.2285
Epoch 164/200
121/121 - 0s - 3ms/step - loss: 15.6076 - val_loss: 8.2267
Epoch 165/200
121/121 - 0s - 2ms/step - loss: 15.5278 - val_loss: 8.2232
Epoch 166/200
121/121 - 0s - 3ms/step - loss: 15.5186 - val_loss: 8.2229
Epoch 167/200
121/121 - 0s - 2ms/step - loss: 15.5207 - val_loss: 8.2220
Epoch 168/200
121/121 - 0s - 2ms/step - loss: 15.6605 - val_loss: 8.2204
Epoch 169/200
121/121 - 0s - 2ms/step - loss: 15.6555 - val_loss: 8.2190
Epoch 170/200
121/121 - 0s - 3ms/step - loss: 15.3211 - val_loss: 8.2179
Epoch 171/200
121/121 - 0s - 2ms/step - loss: 15.5223 - val_loss: 8.2168
Epoch 172/200
121/121 - 0s - 3ms/step - loss: 15.7393 - val_loss: 8.2152
Epoch 173/200
121/121 - 0s - 2ms/step - loss: 15.5104 - val_loss: 8.2140
Epoch 174/200
121/121 - 0s - 2ms/step - loss: 15.7142 - val_loss: 8.2124
Epoch 175/200
121/121 - 0s - 2ms/step - loss: 15.3751 - val_loss: 8.2098
Epoch 176/200
121/121 - 0s - 2ms/step - loss: 15.4906 - val_loss: 8.2105
Epoch 177/200
121/121 - 0s - 3ms/step - loss: 15.3598 - val_loss: 8.2091
Epoch 178/200
121/121 - 0s - 3ms/step - loss: 15.1495 - val_loss: 8.2066
Epoch 179/200
121/121 - 0s - 2ms/step - loss: 15.3605 - val_loss: 8.2060
Epoch 180/200
121/121 - 0s - 3ms/step - loss: 15.3270 - val_loss: 8.2044

```

Epoch 181/200
121/121 - 0s - 2ms/step - loss: 15.1333 - val_loss: 8.2020
Epoch 182/200
121/121 - 0s - 2ms/step - loss: 15.4351 - val_loss: 8.2012
Epoch 183/200
121/121 - 0s - 2ms/step - loss: 15.2774 - val_loss: 8.1993
Epoch 184/200
121/121 - 0s - 3ms/step - loss: 15.1772 - val_loss: 8.1995
Epoch 185/200
121/121 - 0s - 3ms/step - loss: 15.2011 - val_loss: 8.1982
Epoch 186/200
121/121 - 0s - 3ms/step - loss: 15.3532 - val_loss: 8.1984
Epoch 187/200
121/121 - 1s - 5ms/step - loss: 15.0340 - val_loss: 8.1979
Epoch 188/200
121/121 - 0s - 2ms/step - loss: 15.0379 - val_loss: 8.1965
Epoch 189/200
121/121 - 0s - 3ms/step - loss: 15.0680 - val_loss: 8.1950
Epoch 190/200
121/121 - 0s - 2ms/step - loss: 14.9889 - val_loss: 8.1930
Epoch 191/200
121/121 - 0s - 3ms/step - loss: 15.1607 - val_loss: 8.1920
Epoch 192/200
121/121 - 0s - 2ms/step - loss: 15.0040 - val_loss: 8.1920
Epoch 193/200
121/121 - 0s - 2ms/step - loss: 14.9702 - val_loss: 8.1928
Epoch 194/200
121/121 - 0s - 3ms/step - loss: 15.0247 - val_loss: 8.1941
Epoch 195/200
121/121 - 0s - 2ms/step - loss: 14.7758 - val_loss: 8.1931
Epoch 196/200
121/121 - 0s - 2ms/step - loss: 14.8594 - val_loss: 8.1926
Epoch 197/200
121/121 - 0s - 2ms/step - loss: 15.0426 - val_loss: 8.1922
Epoch 198/200
121/121 - 0s - 3ms/step - loss: 14.9661 - val_loss: 8.1916
Epoch 199/200
121/121 - 0s - 2ms/step - loss: 14.9118 - val_loss: 8.1912
Epoch 200/200
121/121 - 0s - 3ms/step - loss: 14.7804 - val_loss: 8.1902
Restoring model weights from the end of the best epoch: 200.

```

```
Out[12]: <keras.src.callbacks.history.History at 0x7ef1a0703690>
```

This is how our 'tuned' model design will look, with a total of 2 hidden layers (with 6 and 21 units) and a final output layer:

```
In [13]: model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_5 (Dense)	(None, 11)	55
dropout_4 (Dropout)	(None, 11)	0
dense_6 (Dense)	(None, 21)	252
dropout_5 (Dropout)	(None, 21)	0
dense_7 (Dense)	(None, 1)	22

Total params: 989 (3.87 KB)

Trainable params: 329 (1.29 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 660 (2.58 KB)

And here we will end the deep learning part of our task today! We have built a somewhat simple model using hyper-parameter tuning. Now, let's see whether the predictions (out-of-sample, of course) of this model are actually useful in a trading strategy for the test period selected.

3. Financial performance of the model

Now, we will check if the predictions delivered in the test sample are valid for developing a trading strategy!

3.1 Backtesting out-of-sample (in test sample)

As usual, we will now perform a backtesting of a trading strategy based on the predictions of the model:

```
In [14]: y_pred = model.predict(X_test)
```

101/101 ————— 0s 925us/step

```
In [15]: df_predictions = pd.DataFrame(
    {
        'Date': test_time, # No flatten needed for DatetimeIndex
        'Pred': y_pred.flatten() if y_pred.ndim > 1 else y_pred, # Flatten if 2
        'Ret': Ret_vector.flatten() if Ret_vector.ndim > 1 else Ret_vector, # F
    }
)
df_predictions.tail()
df = df_predictions
df.tail()
```

Out[15]:

	Date	Pred	Ret
3199	2025-03-25	1.448421	1.37
3200	2025-03-26	1.211321	-0.99
3201	2025-03-27	1.418115	1.05
3202	2025-03-28	1.219985	-2.66
3203	2025-03-31	1.213160	1.94

As we have done other times before, we will compute the returns of 3 strategies in the test period:

- **Buy and hold:** simply buys the stock at the beginning of the test period and holds it until the end.
- **Long-only:** this strategy either goes long in the stock (if the predicted 25-day return is positive) or goes to cash (with a 0% return associated).
- **Long/Short** this strategy goes long (short) in the stock when the 25-day ahead return predicted by the model is positive (negative)

```
In [16]: df['Ret'] = df['Ret'] / 100

df["Positions"] = np.where(df["Pred"] > 0, 1, -1)
df["Strat_ret"] = df["Positions"].shift(1) * df["Ret"]
df["Positions_L"] = df["Positions"].shift(1)
df["Positions_L"][df["Positions_L"] == -1] = 0
df["Strat_ret_L"] = df["Positions_L"] * df["Ret"]
df["CumRet"] = df["Strat_ret"].expanding().apply(lambda x: np.prod(1 + x) - 1)
df["CumRet_L"] = df["Strat_ret_L"].expanding().apply(lambda x: np.prod(1 + x) - 1)
df["bhRet"] = df["Ret"].expanding().apply(lambda x: np.prod(1 + x) - 1)

Final_Return_L = np.prod(1 + df["Strat_ret_L"]) - 1
Final_Return = np.prod(1 + df["Strat_ret"]) - 1
Buy_Return = np.prod(1 + df["Ret"]) - 1

print("Strat Return Long Only =", Final_Return_L * 100, "%")
print("Strat Return =", Final_Return * 100, "%")
print("Buy and Hold Return =", Buy_Return * 100, "%")
```

/tmp/ipykernel_13387/2786902323.py:6: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df["Positions_L"][df["Positions_L"] == -1] = 0
```

Strat Return Long Only = 739.5384870082764 %

Strat Return = 407.82058285920135 %

Buy and Hold Return = 1177.106632396256 %

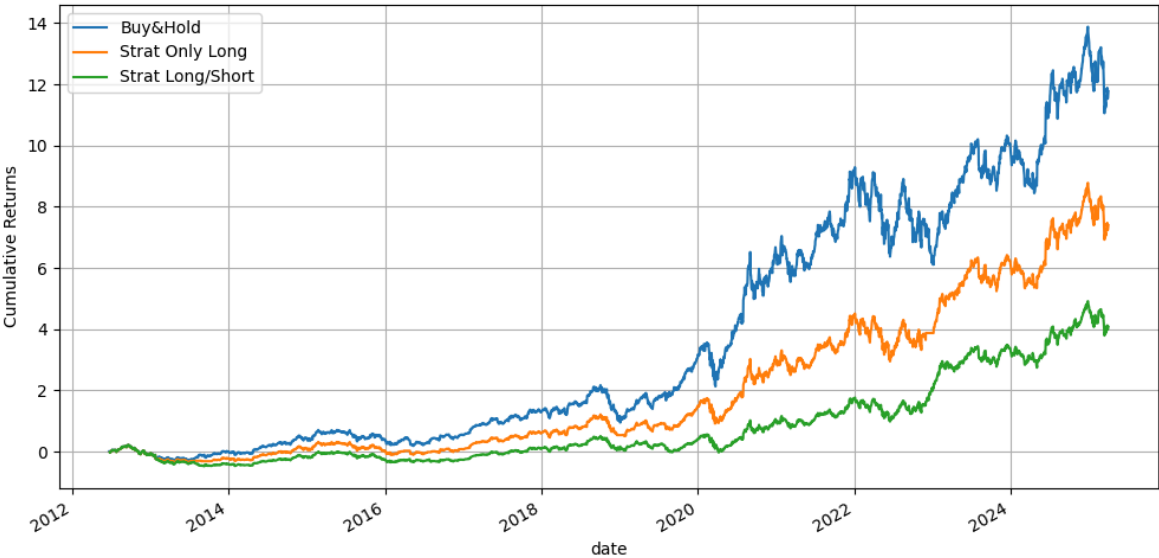
Judging by the numbers obtained here, it seems that, under these tuned hyper-parameters and, remember, for the testing period of our choice, our model does a fairly

decent job in timing the AAPL stock. Let's better see it graphically:

```
In [17]: import matplotlib.pyplot as plt

fig = plt.figure(figsize=(12, 6))
ax = plt.gca()
df.plot(x="Date", y="bhRet", label="Buy&Hold", ax=ax)
df.plot(x="Date", y="CumRet_L", label="Strat Only Long", ax=ax)
df.plot(x="Date", y="CumRet", label="Strat Long/Short", ax=ax)
plt.xlabel("date")
plt.ylabel("Cumulative Returns")
plt.grid()
plt.show()

df.describe()
```



Out[17]:

	Date	Pred	Ret	Positions	Strat_ret	Positior
count	3204	3204.000000	3204.000000	3204.000000	3203.000000	3203.000
mean	2018-11-08 16:05:23.595505664	1.702753	0.000952	0.859551	0.000664	0.929
min	2012-06-22 00:00:00	-1.788622	-0.128600	-1.000000	-0.128600	0.000
25%	2015-08-31 18:00:00	0.736493	-0.007500	1.000000	-0.007700	1.000
50%	2018-11-06 12:00:00	1.479906	0.001000	1.000000	0.000700	1.000
75%	2022-01-13 06:00:00	2.439269	0.010300	1.000000	0.010000	1.000
max	2025-03-31 00:00:00	8.799627	0.119800	1.000000	0.119800	1.000
std	NaN	1.450142	0.017669	0.511130	0.017684	0.255



4. Conclusion

In this last lesson of Module 1, we have implemented a basic tuning of model hyperparameters. Even though we have performed a somewhat simple tuning that focused on very few hyper-parameters, there are 2 things that we have likely come to realize: (i) tuning is computationally costly, and (ii) it can actually make a huge difference!. In the next module, we will continue to explore more advanced Deep Learning models, that will allow us to gradually increase the complexity of the tasks at hand and its implementation.

See you there!

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.